Dependable Distributed Systems
Master of Science in Engineering in Computer Science

AA 2026/2027

LECTURE 7A: FAILURE DETECTION

Recap on Timing Assumptions

Synchronous

- timing assumptions are explicit on
  - Upper bounds on process execution time
  - Upper bounds on communication time
  - Existence of a common global clock

Asynchronous

- there are no timing assumptions

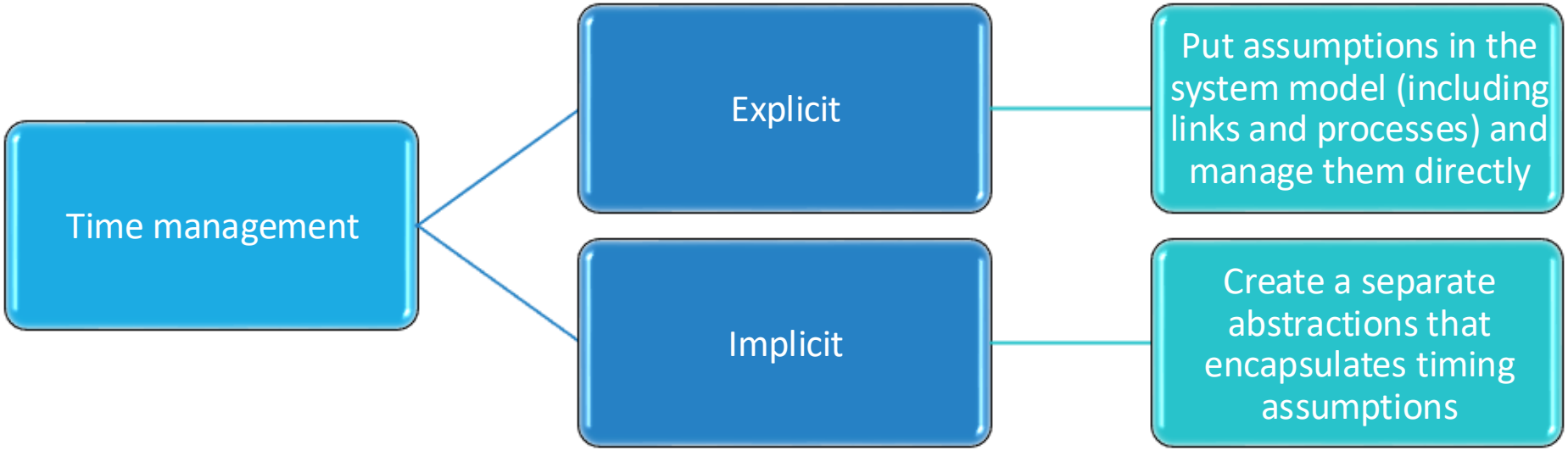
Partial synchrony

- requires abstract timing assumptions (after an unknown time t the system becomes synchronous)

Recap on Timing Assumptions

NOTE

- manipulating time inside a protocol/algorithm is complex and the correctness proof may become very involved and sometimes prone to errors



Failure Detector Abstraction

A Failure Detector is an oracle providing information about the failure state of a process

- it is a software module to be used together with process and link abstractions
- It encapsulates timing assumptions of either partially synchronous or fully synchronous system
  - Stronger are the timing assumption, more accurate the information provided by a failure detector will be

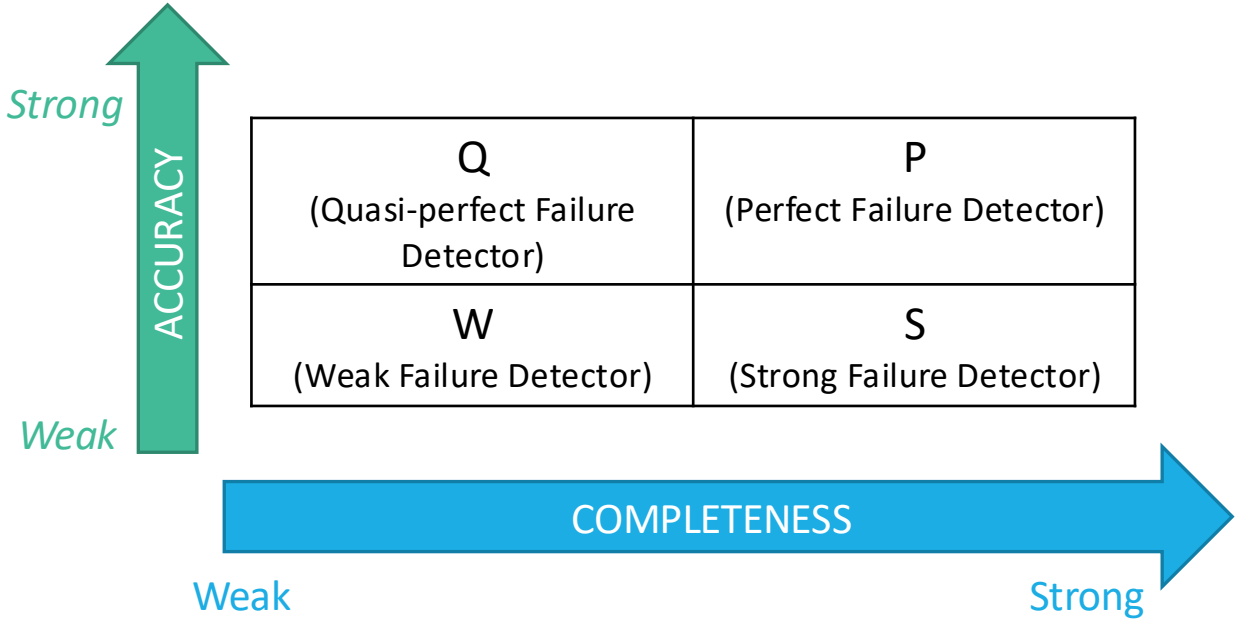
A failure detector is generally described by two properties:

- Accuracy (informally is the ability to avoid mistakes in the detection)
- Completeness (informally is ability to detect all failures)

Failure Detectors Classification

Correct processes are never suspected

Some correct process is never suspected



Eventually every process that crashes is permanently suspected by **some** correct process

Eventually every process that crashes is permanently suspected by **every** correct process

Failure Detectors Classification

OBSERVATION

- W guarantees that there is at least one correct process that is never suspected.
- Practically speaking, this could be difficult to achieve and thus it is worth to consider accuracy properties to hold eventually

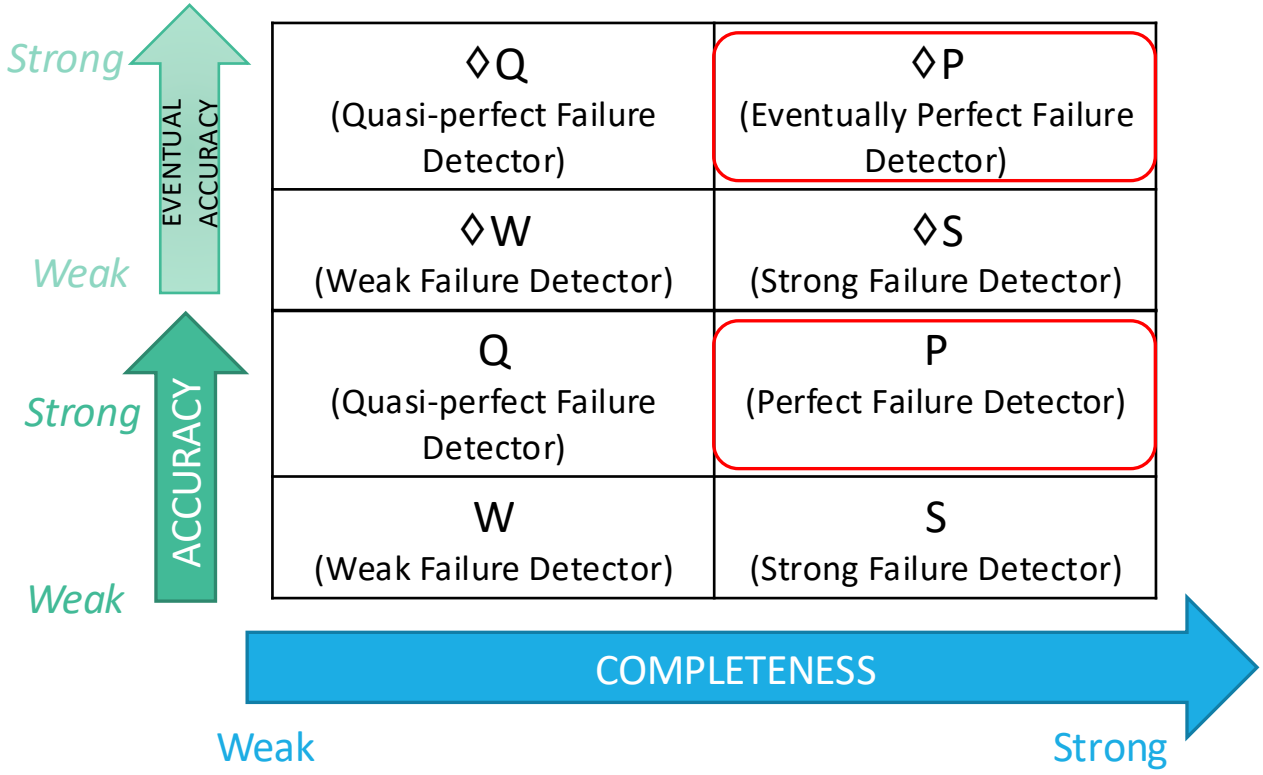
EVENTUAL STRONG ACCURACY

- There is a time after which correct processes are not suspected

EVENTUAL WEAK ACCURACY

- There is a time after which some correct process in not suspected

Failure Detectors Classification



Perfect Failure detectors (P)

System model

- synchronous system
- crash failures

Using its own clock and the bounds of the synchrony model, a process can infer if another process has crashed

Perfect failure detectors (P) Specification

Module 2.6: Interface and properties of the perfect failure detector

Module:

Name: PerfectFailureDetector, **instance** $\mathcal{P}$.

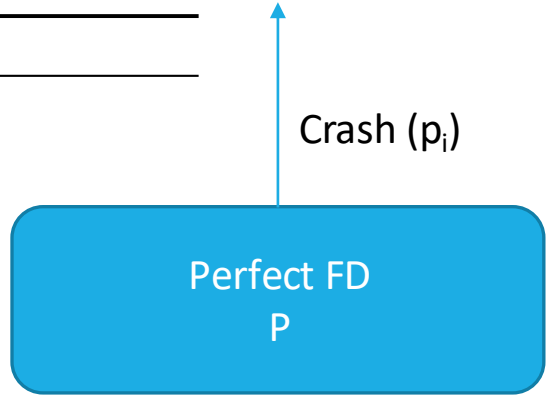
Events:

Indication: $\langle\, \mathcal{P}, \mathit{Crash} \mid p \,\rangle$: Detects that process p has crashed.

Properties:

PFD1: *Strong completeness:* Eventually, every process that crashes is permanently detected by every correct process.

PFD2: *Strong accuracy:* If a process p is detected by any process, then p has crashed.



Perfect failure detectors (P) Implementation

Algorithm 2.5: Exclude on Timeout

Implements:
PerfectFailureDetector, **instance** $\mathcal{P}$.

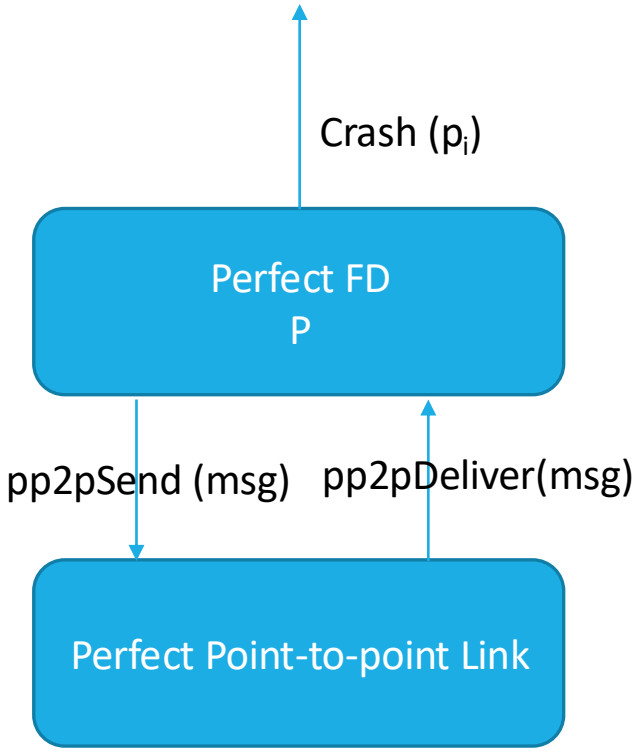
Uses:
PerfectPointToPointLinks, **instance** *pl*.

upon event $\langle\ \mathcal{P}, \mathit{Init}\ \rangle$ **do**
  $\mathit{alive} := \Pi$;
  $\mathit{detected} := \emptyset$;
  starttimer(Δ);

upon event $\langle\ \mathit{Timeout}\ \rangle$ **do**
  forall $p \in \Pi$ **do**
    if $(p \notin \mathit{alive}) \wedge (p \notin \mathit{detected})$ **then**
      $\mathit{detected} := \mathit{detected} \cup \{p\}$;
      trigger $\langle\ \mathcal{P}, \mathit{Crash} \mid p\ \rangle$;
    trigger $\langle\ pl, \mathit{Send} \mid p, [\text{HEARTBEATREQUEST}]\ \rangle$;
  $\mathit{alive} := \emptyset$;
  starttimer(Δ);

upon event $\langle\ pl, \mathit{Deliver} \mid q, [\text{HEARTBEATREQUEST}]\ \rangle$ **do**
  trigger $\langle\ pl, \mathit{Send} \mid q, [\text{HEARTBEATREPLY}]\ \rangle$;

upon event $\langle\ pl, \mathit{Deliver} \mid p, [\text{HEARTBEATREPLY}]\ \rangle$ **do**
  $\mathit{alive} := \mathit{alive} \cup \{p\}$;



Correctness

- To prove the correctness, we must prove that both Strong Completeness and Strong Accuracy are satisfied

- What if links are fair loss?

- What if we select a timeout too long?

- What if we select a timeout too short?

Eventually perfect failure detectors (◊P)

System model

- partial synchrony
- Crash failures
- Perfect point-to-point links

Crashes can be accurately detected only after a (unknown) time *t*

- Before time *t* the systems behaves as an asynchronous one
- The failure detector may make mistake before time *t* considering correct processes as crashed.
- The notion of detection becomes suspicious

Eventually perfect failure detectors (◊P) Specification

Module 2.8: Interface and properties of the eventually perfect failure detector

Module:

Name: EventuallyPerfectFailureDetector, **instance** $\diamond\mathcal{P}$.

Events:

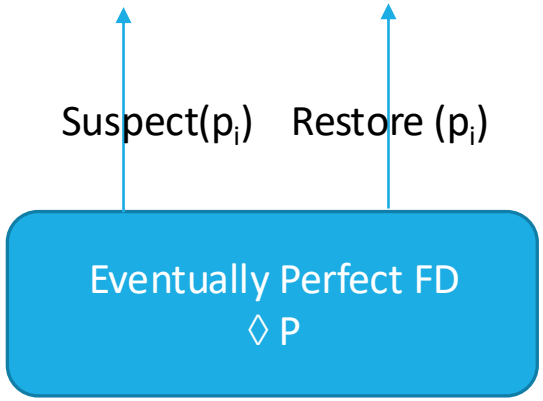
Indication: $\langle\ \diamond\mathcal{P}, \mathit{Suspect} \mid p\ \rangle$: Notifies that process p is suspected to have crashed.

Indication: $\langle\ \diamond\mathcal{P}, \mathit{Restore} \mid p\ \rangle$: Notifies that process p is not suspected anymore.

Properties:

EPFD1: *Strong completeness:* Eventually, every process that crashes is permanently suspected by every correct process.

EPFD2: *Eventual strong accuracy:* Eventually, no correct process is suspected by any correct process.



Basic constructions rules of an eventually perfect FD

Use timeouts to suspect processes that did not sent expected messages

A suspect may be wrong

- A process p_i may suspect another one p_j as the current timeout is too short

◊ P is ready to change its judgment as soon as it receives a message from p_j

- In this case, the timeout value is updated

If p_j has actually crashed, p_i does not change its judgment anymore

Eventually perfect failure detectors (◊P) Implementation

Algorithm 2.7: Increasing Timeout

Implements:
EventuallyPerfectFailureDetector, **instance** $\diamond\mathcal{P}$.

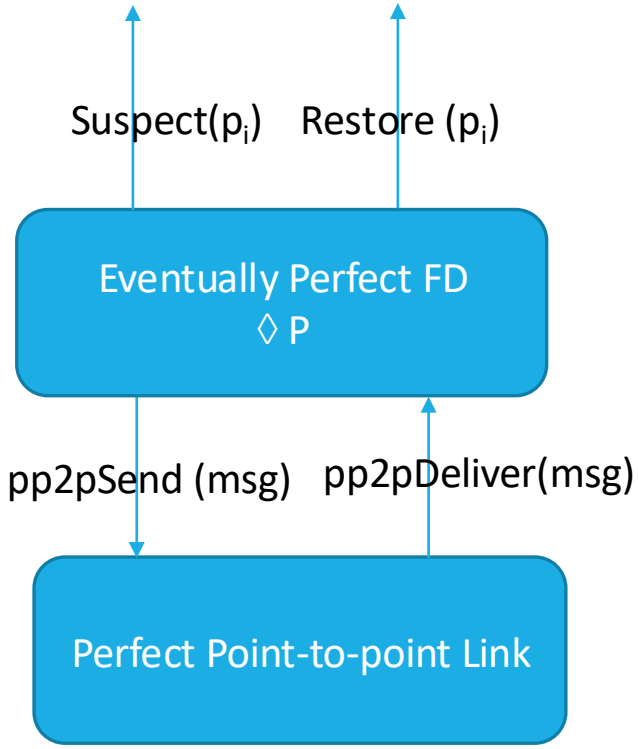
Uses:
PerfectPointToPointLinks, **instance** *pl*.

upon event $\langle\ \diamond\mathcal{P}, \mathit{Init}\ \rangle$ **do**
  $\mathit{alive} := \Pi$;
  $\mathit{suspected} := \emptyset$;
  $\mathit{delay} := \Delta$;
  starttimer(*delay*);

upon event $\langle\ \mathit{Timeout}\ \rangle$ **do**
  if $\mathit{alive} \cap \mathit{suspected} \neq \emptyset$ **then**
    $\mathit{delay} := \mathit{delay} + \Delta$;
  forall $p \in \Pi$ **do**
    if $(p \notin \mathit{alive}) \wedge (p \notin \mathit{suspected})$ **then**
      $\mathit{suspected} := \mathit{suspected} \cup \{p\}$;
      trigger $\langle\ \diamond\mathcal{P}, \mathit{Suspect} \mid p\ \rangle$;
    else if $(p \in \mathit{alive}) \wedge (p \in \mathit{suspected})$ **then**
      $\mathit{suspected} := \mathit{suspected} \setminus \{p\}$;
      trigger $\langle\ \diamond\mathcal{P}, \mathit{Restore} \mid p\ \rangle$;
    trigger $\langle\ \mathit{pl}, \mathit{Send} \mid p, [\text{HEARTBEATREQUEST}]\ \rangle$;
  alive := ∅;
  starttimer(*delay*);

upon event $\langle\ \mathit{pl}, \mathit{Deliver} \mid q, [\text{HEARTBEATREQUEST}]\ \rangle$ **do**
  trigger $\langle\ \mathit{pl}, \mathit{Send} \mid q, [\text{HEARTBEATREPLY}]\ \rangle$;

upon event $\langle\ \mathit{pl}, \mathit{Deliver} \mid p, [\text{HEARTBEATREPLY}]\ \rangle$ **do**
  $\mathit{alive} := \mathit{alive} \cup \{p\}$;



Correctness

Strong completeness

- If a process crashes, it will stop to send messages. Therefore the process will be suspected by any correct process and no process will revise the judgement.

Eventual strong accuracy

- After time T the system becomes synchronous. i.e., after that time a message sent by a correct process p to another one q will be delivered within a bounded time. If p was wrongly suspected by q, then q will revise its suspicious.

References

C. Cachin, R. Guerraoui and L. Rodrigues. Introduction to Reliable and Secure Distributed Programming, Springer, 2011

- Chapter 2 – from Section 2.6.1 to Section 2.6.5

T. Chandra, S. Toueg *Unreliable Failure Detectors for Reliable Distributed Systems*

- https://dl.acm.org/doi/pdf/10.1145/226643.226647

Dependable Distributed Systems
Master of Science in Engineering in Computer Science

AA 2026/2027

LECTURE 7B: LEADER ELECTION

Recap on Timing Assumptions

Synchronous

- 兮 timing assumptions are explicit either on
  - 兮 Bounds on process executions and communication channels, or
  - 兮 Existence of a common global clock, or
  - 兮 Both

Asynchronous

- 兮 there are no timing assumptions

Recap on Timing Assumptions

Partial synchrony requires abstract timing assumptions (after an unknown time t the system becomes synchronous)

Two choices:

1. Put assumption on the system model (including links and processes)
2. Create a separate abstractions that encapsulates those timing assumptions

Note: manipulating time inside a protocol/algorithm is complex and the correctness proof may become very involved and sometimes prone to errors

An alternative

Sometimes, we may be interested in knowing one process that is alive instead of monitoring failures

- E.g., Need of a coordinator

We can use a different oracle (called *leader election* module) that reports a process that is alive

Leader Election Specification

Module 2.7: Interface and properties of leader election

Module:

Name: LeaderElection, **instance** *le*.

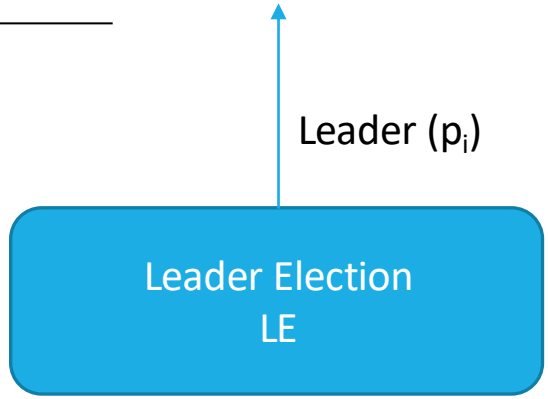
Events:

Indication: ⟨ *le*, *Leader* | *p* ⟩: Indicates that process *p* is elected as leader.

Properties:

LE1: *Eventual detection:* Either there is no correct process, or some correct process is eventually elected as the leader.

LE2: *Accuracy:* If a process is leader, then all previously elected leaders have crashed.



Leader Election Implementation

Algorithm 2.6: Monarchical Leader Election

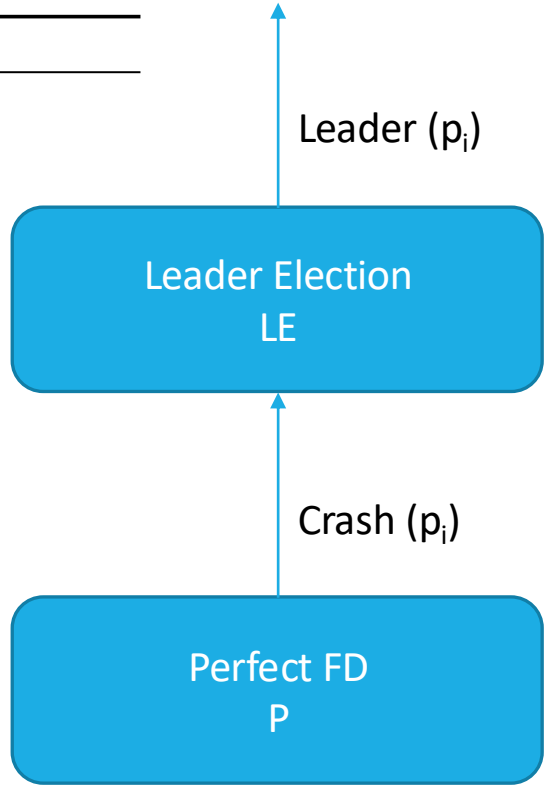
Implements:
LeaderElection, **instance** *le*.

Uses:
PerfectFailureDetector, **instance** $\mathcal{P}$.

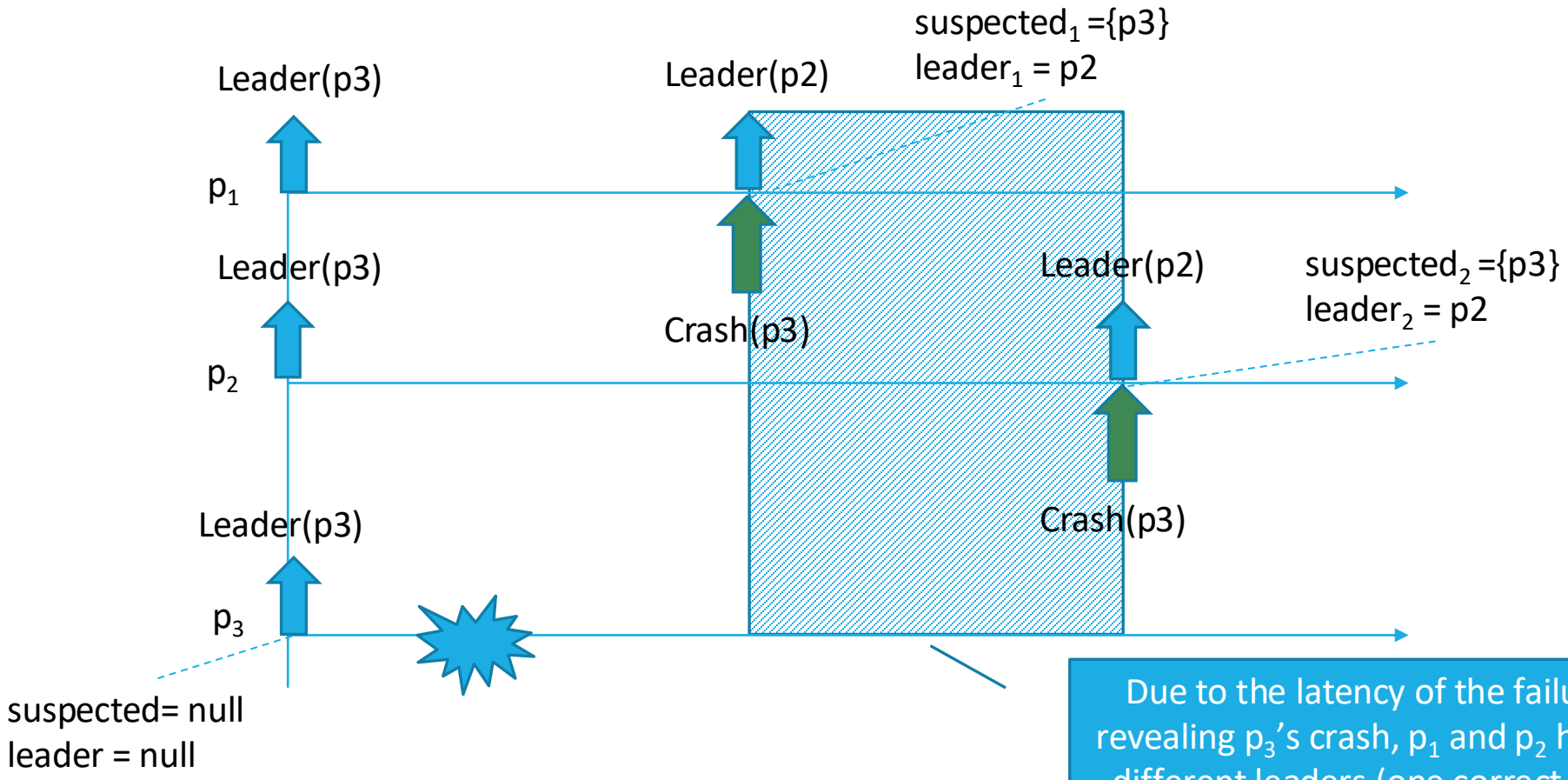
upon event $\langle$ *le*, *Init* $\rangle$ **do**
$suspected := \emptyset$;
$leader := \perp$;

upon event $\langle$ $\mathcal{P}$, *Crash* | p $\rangle$ **do**
$suspected := suspected \cup \{p\}$;

upon $leader \neq maxrank(\Pi \setminus suspected)$ **do**
$leader := maxrank(\Pi \setminus suspected)$;
trigger $\langle$ *le*, *Leader* | *leader* $\rangle$;

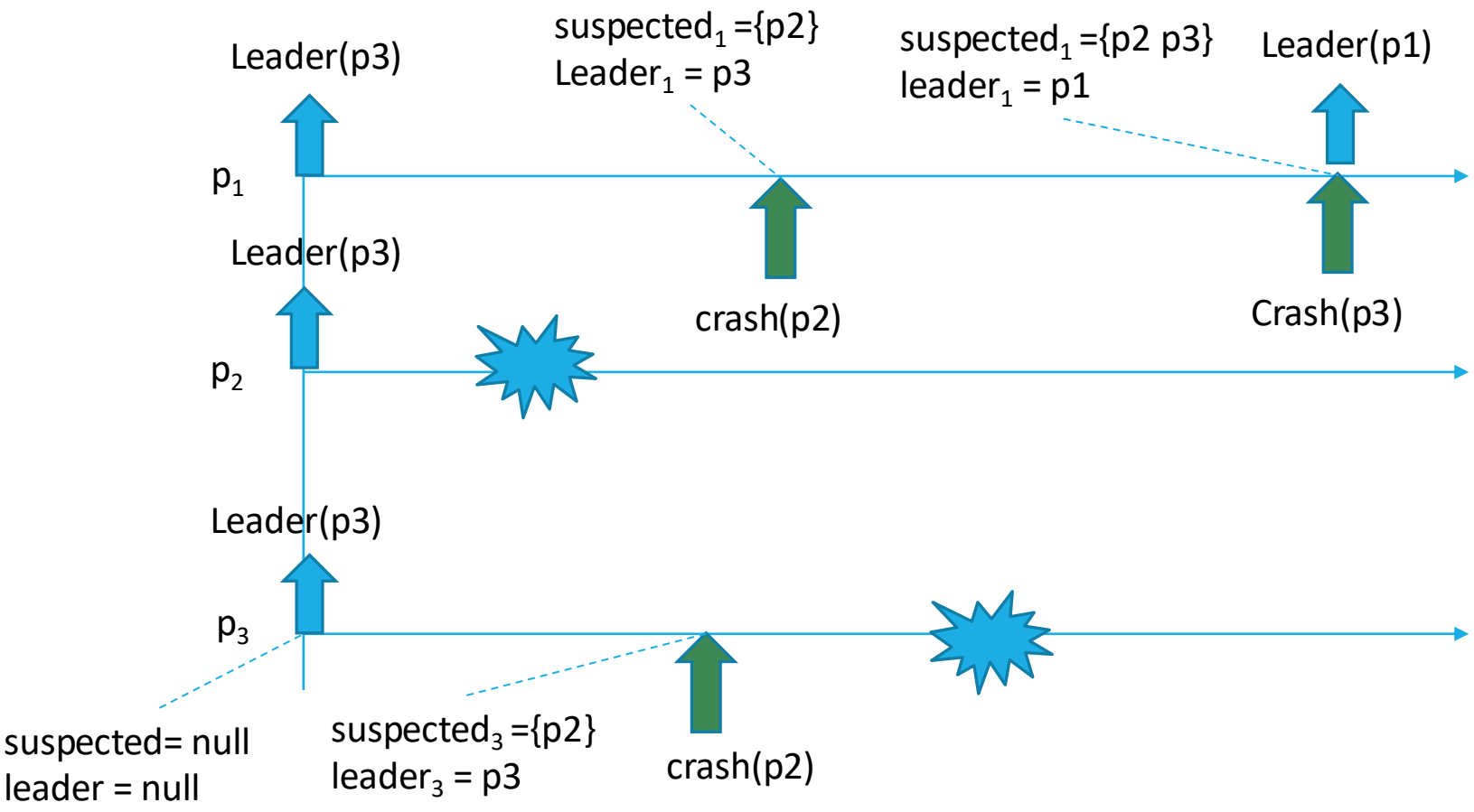


Example

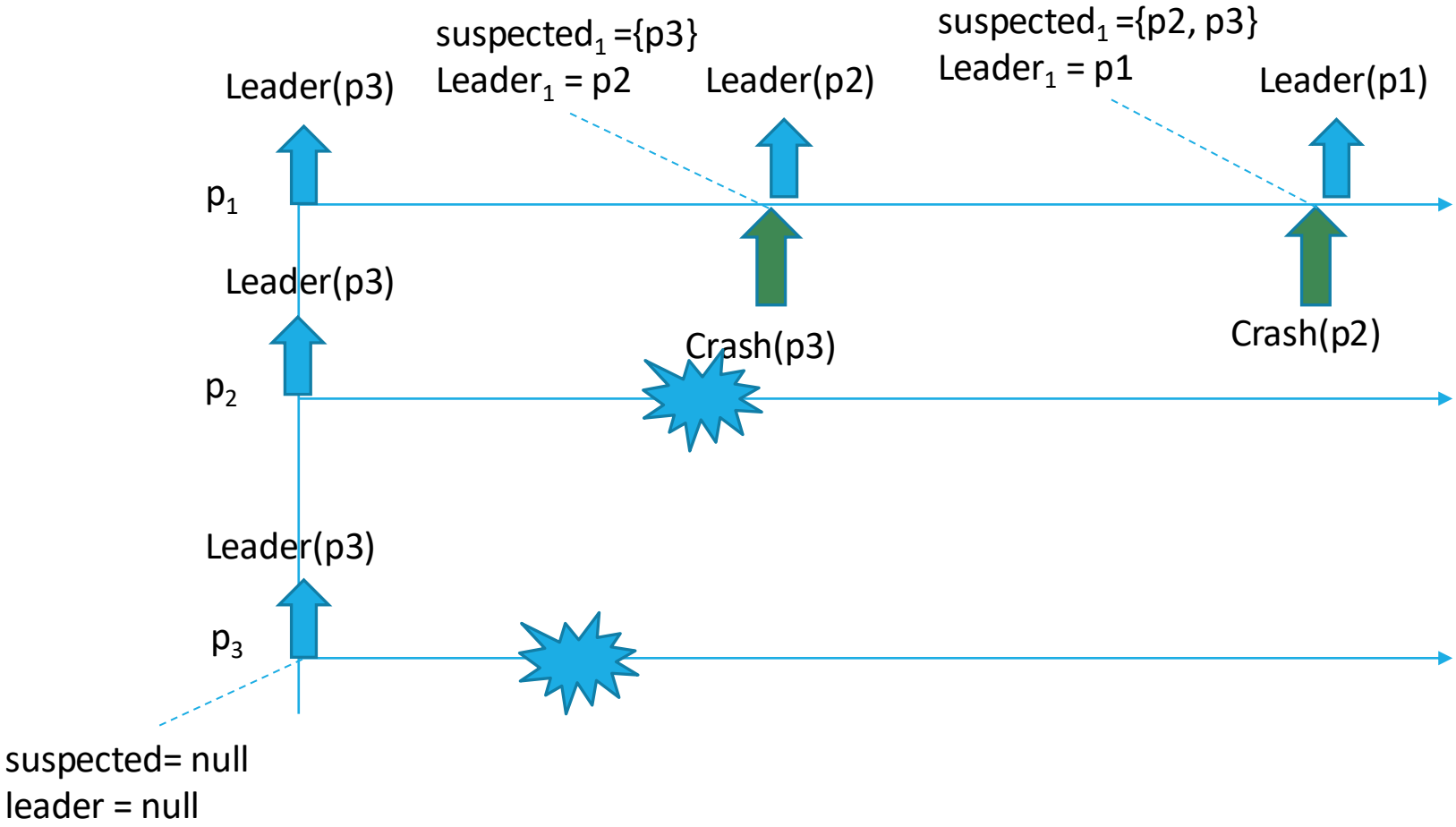


Due to the latency of the failure detector in revealing p_3's crash, p_1 and p_2 have temporarily different leaders (one correct and one faulty)

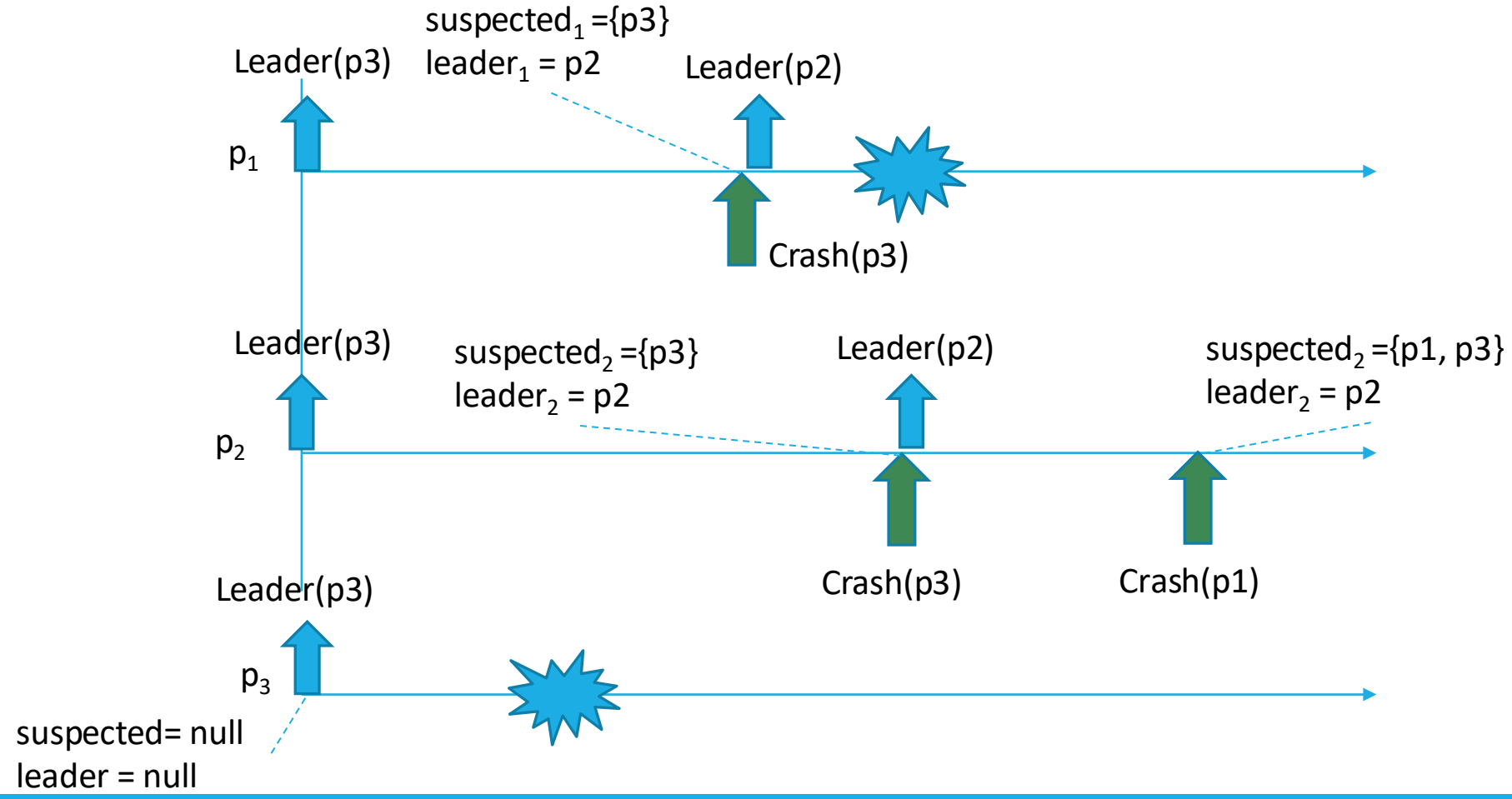
Example 2


Leader(p3)
suspected$_1$ ={p2}
Leader$_1$ = p3
suspected$_1$ ={p2 p3}
leader$_1$ = p1
Leader(p1)
p$_1$
Leader(p3)
crash(p2)
Crash(p3)
p$_2$
Leader(p3)
p$_3$
suspected= null
leader = null
suspected$_3$ ={p2}
leader$_3$ = p3
crash(p2)

Example 3


suspected$_1$ ={p3}
Leader$_1$ = p2
suspected$_1$ ={p2, p3}
Leader$_1$ = p1
Leader(p3)
Leader(p2)
Leader(p1)
p$_1$
Leader(p3)
Crash(p3)
Crash(p2)
p$_2$
Leader(p3)
p$_3$
suspected= null
leader = null

Example 4


suspected$_1$ ={p3}
leader$_1$ = p2
Leader(p3)
Leader(p2)
p_1
Crash(p3)
Leader(p3)
suspected$_2$ ={p3}
leader$_2$ = p2
Leader(p2)
suspected$_2$ ={p1, p3}
leader$_2$ = p2
p_2
Crash(p3)
Crash(p1)
Leader(p3)
p_3
suspected= null
leader = null

Correctness

What if the Failure detector is not perfect?

Eventual leader election (Ω)

Module 2.9: Interface and properties of the eventual leader detector

Module:

Name: EventualLeaderDetector, **instance** Ω.

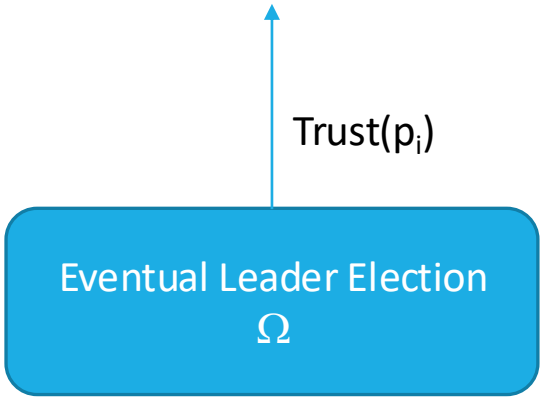
Events:

Indication: $\langle\ \Omega, Trust \mid p\ \rangle$: Indicates that process p is trusted to be leader.

Properties:

ELD1: *Eventual accuracy:* There is a time after which every correct process trusts some correct process.

ELD2: *Eventual agreement:* There is a time after which no two correct processes trust different correct processes.



Observation on Ω

Ω ensures that *eventually* correct processes will elect the same correct process as their leader

Ω does not guarantee that

- Leaders change in an arbitrary manner and for an arbitrary period of time
- many leaders might be elected during the same period of time without having crashed

Once a unique leader is determined, and does not change again, we say that the leader has *stabilized*

Eventual leader election (Ω)

Using Crash-stop process abstraction

- Obtained directly by <>P by using a deterministic rule on processes that are not suspected by <>P
- trust the process with the highest identifier among all processes that are not suspected by <>P

IMPORTANT ASSUMPTION

- There always exists at least one correct process (otherwise Ω cannot be built)

Ω Implementation

Algorithm 2.8: Monarchical Eventual Leader Detection

Implements:
EventualLeaderDetector, **instance** Ω.

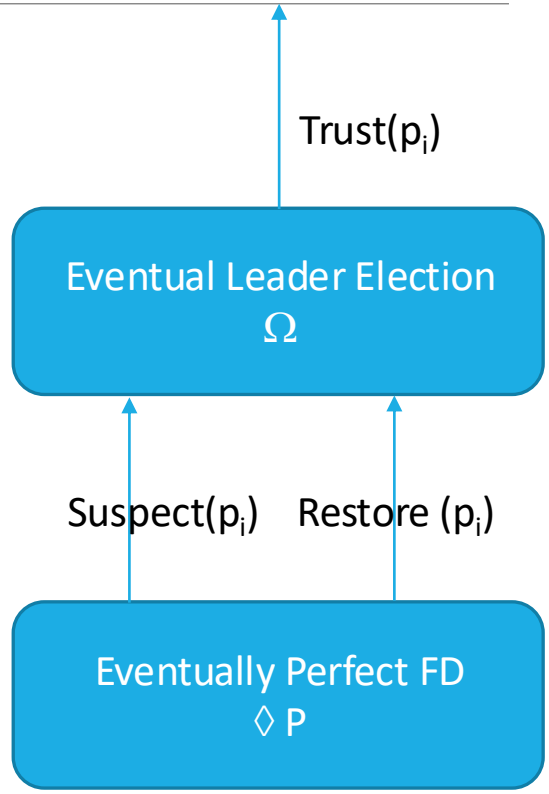
Uses:
EventuallyPerfectFailureDetector, **instance** $\Diamond\mathcal{P}$.

upon event $\langle\ \Omega, Init\ \rangle$ **do**
$suspected := \emptyset$;
$leader := \perp$;

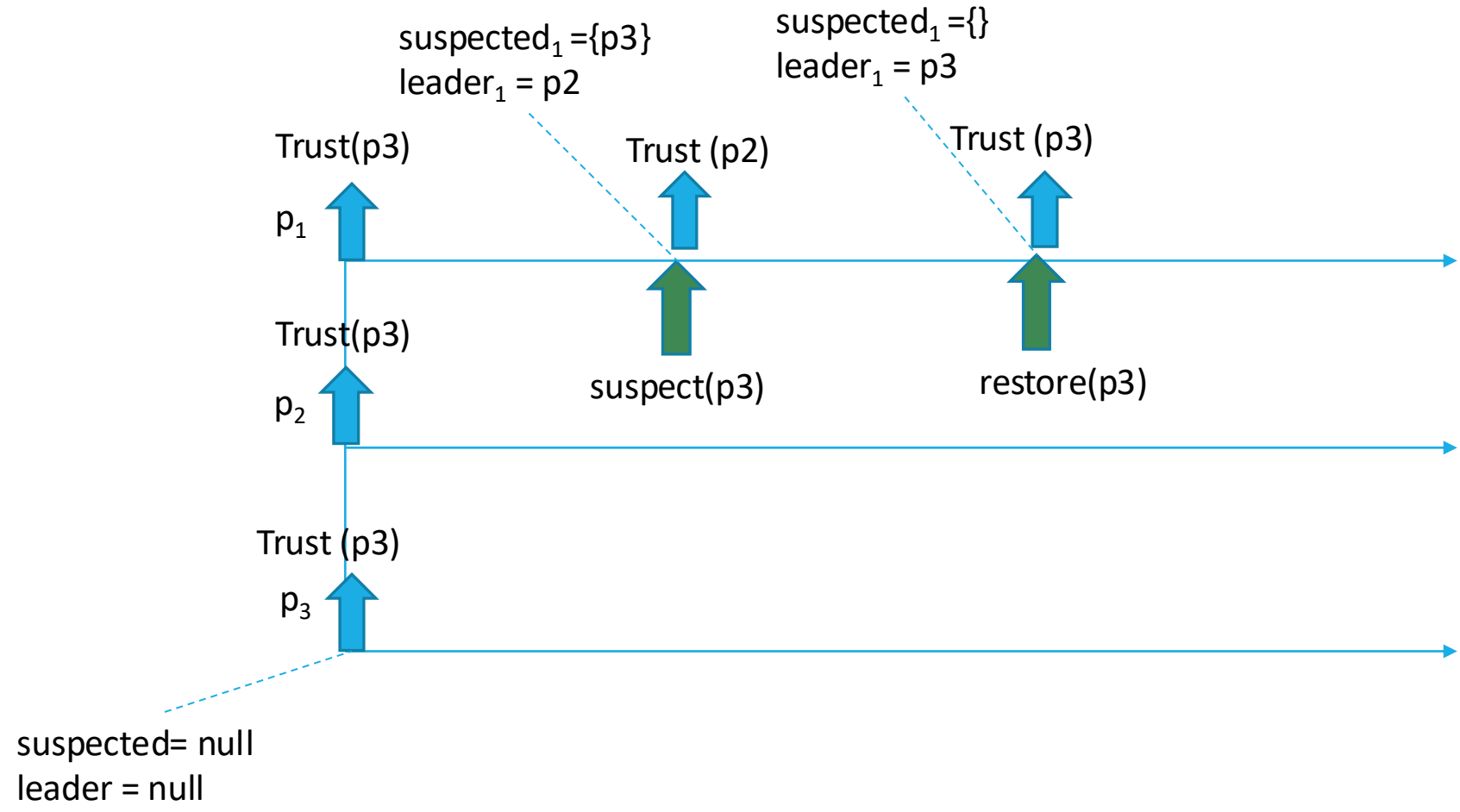
upon event $\langle\ \Diamond\mathcal{P}, Suspect \mid p\ \rangle$ **do**
$suspected := suspected \cup \{p\}$;

upon event $\langle\ \Diamond\mathcal{P}, Restore \mid p\ \rangle$ **do**
$suspected := suspected \setminus \{p\}$;

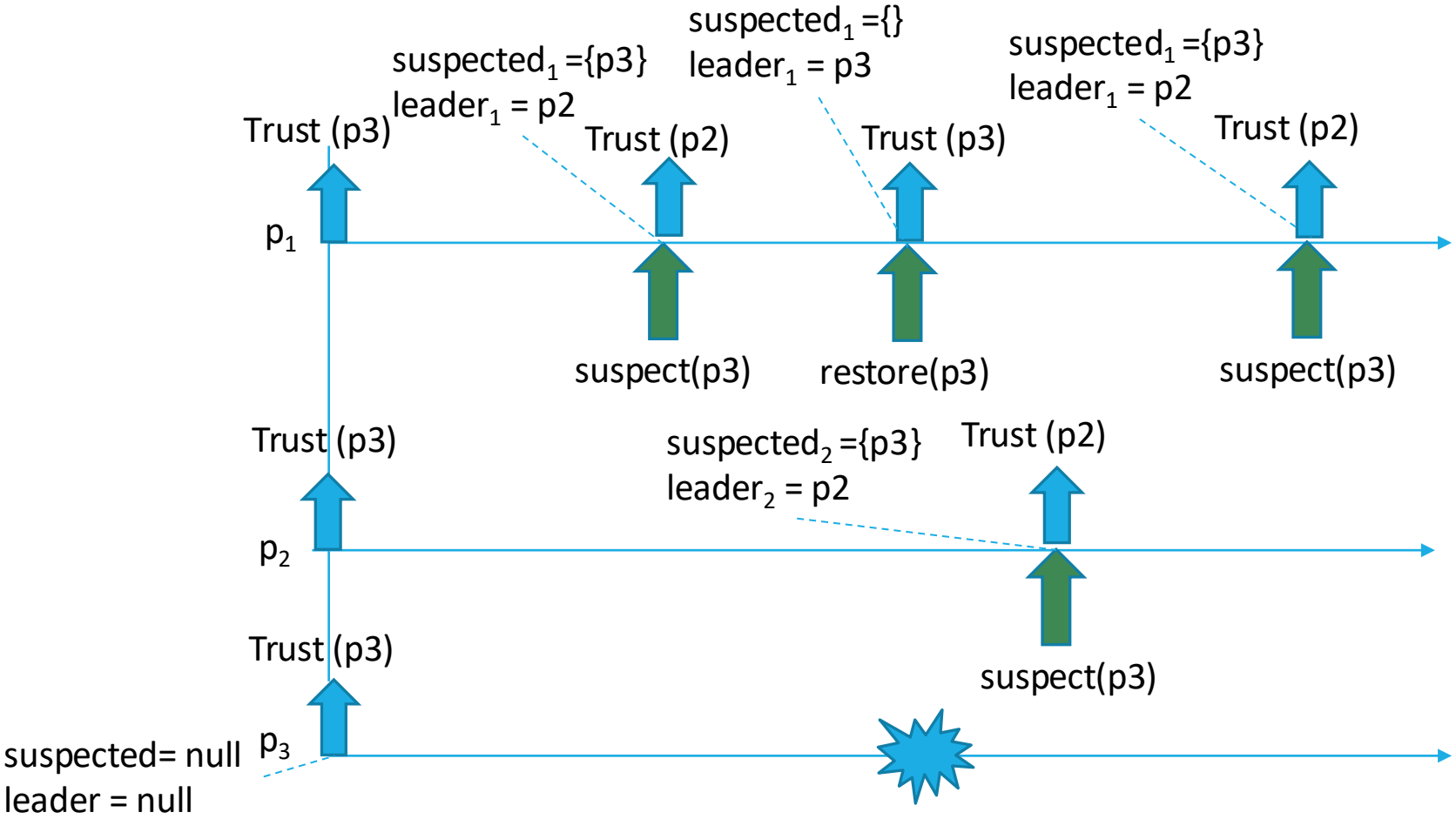
upon $leader \neq maxrank(\Pi \setminus suspected)$ **do**
$leader := maxrank(\Pi \setminus suspected)$;
trigger $\langle\ \Omega, Trust \mid leader\ \rangle$;



Example


suspected$_1$ ={p3}
leader$_1$ = p2
suspected$_1$ ={}
leader$_1$ = p3
Trust(p3)
Trust (p2)
Trust (p3)
p_1
Trust(p3)
p_2
suspect(p3)
restore(p3)
Trust (p3)
p_3
suspected= null
leader = null

Example 2


suspected$_1$ ={}
leader$_1$ = p3
suspected$_1$ ={p3}
leader$_1$ = p2
suspected$_1$ ={p3}
leader$_1$ = p2
Trust (p3)
Trust (p2)
Trust (p3)
Trust (p2)
p$_1$
suspect(p3)
restore(p3)
suspect(p3)
Trust (p3)
suspected$_2$ ={p3}
leader$_2$ = p2
Trust (p2)
p$_2$
Trust (p3)
suspect(p3)
suspected= null
leader = null
p$_3$

Eventual leader election (Ω)

System model

- Crash-Recovery
- Partial synchrony

Under this assumption, a correct process means:

1. A process that does not crash or
2. A process that crashes, eventually recovers and never crashes again

Ω With crash-recovery, fair lossy links and timeouts

Algorithm 2.9: Elect Lower Epoch

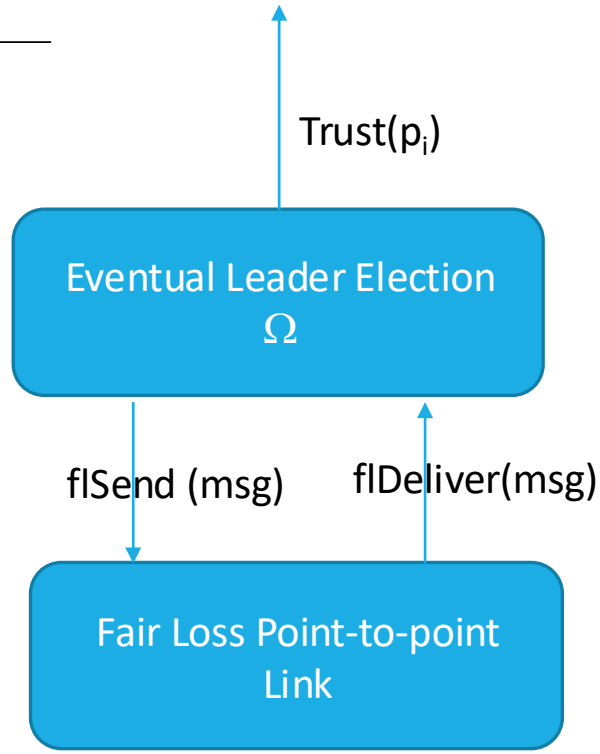
Implements:
EventualLeaderDetector, **instance** Ω.

Uses:
FairLossPointToPointLinks, **instance** *fll*.

upon event $\langle\ \Omega, Init\ \rangle$ **do**
epoch := 0;
store(*epoch*);
candidates := $\emptyset$;
trigger $\langle\ \Omega, Recovery\ \rangle$;

keeps track of how many times the process crashed and recovered

upon event $\langle\ \Omega, Recovery\ \rangle$ **do**
leader := maxrank(Π);
trigger $\langle\ \Omega, Trust \mid leader\ \rangle$;
delay := Δ;
retrieve(*epoch*);
epoch := *epoch* + 1;
store(*epoch*);
forall $p \in \Pi$ **do**
trigger $\langle\ fll, Send \mid p, [\text{HEARTBEAT}, epoch]\ \rangle$;
candidates := $\emptyset$;
starttimer(*delay*);



Ω With crash-recovery, fair lossy links and timeouts

Algorithm 2.9: Elect Lower Epoch

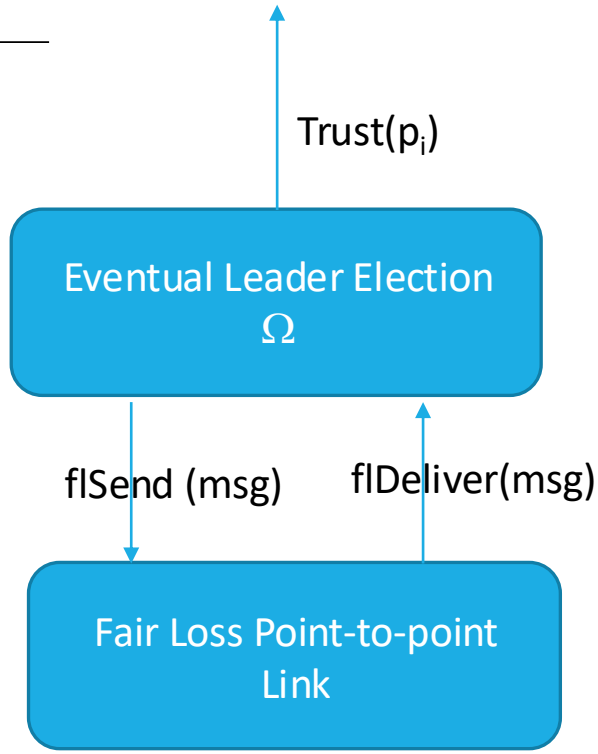
Implements:
EventualLeaderDetector, **instance** Ω.

Uses:
FairLossPointToPointLinks, **instance** *fll*.

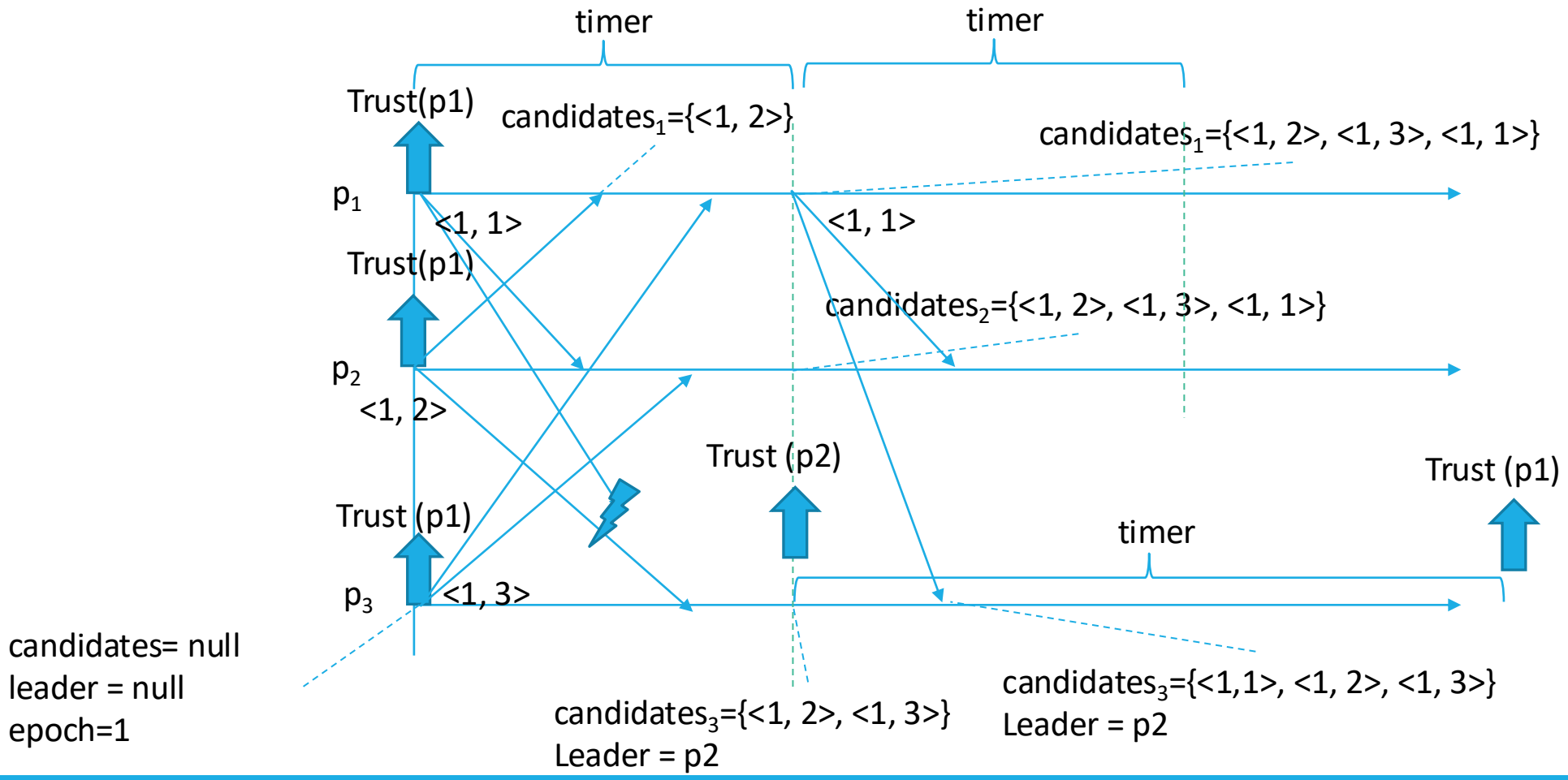
upon event $\langle$ *Timeout* $\rangle$ **do**
  newleader := *select*(*candidates*);
  if *newleader* $\neq$ *leader* **then**
    delay := *delay* + Δ;
    leader := *newleader*;
    trigger $\langle$ Ω, *Trust* | *leader* $\rangle$;
  forall $p \in \Pi$ **do**
    trigger $\langle$ *fll*, *Send* | p, [HEARTBEAT, *epoch*] $\rangle$;
  candidates := $\emptyset$;
  starttimer(*delay*);

upon event $\langle$ *fll*, *Deliver* | q, [HEARTBEAT, ep] $\rangle$ **do**
  if exists $(s, e) \in$ *candidates* such that $s = q \wedge e < ep$ **then**
    candidates := *candidates* $\setminus \{(q, e)\}$;
  candidates := *candidates* $\cup (q, ep)$;

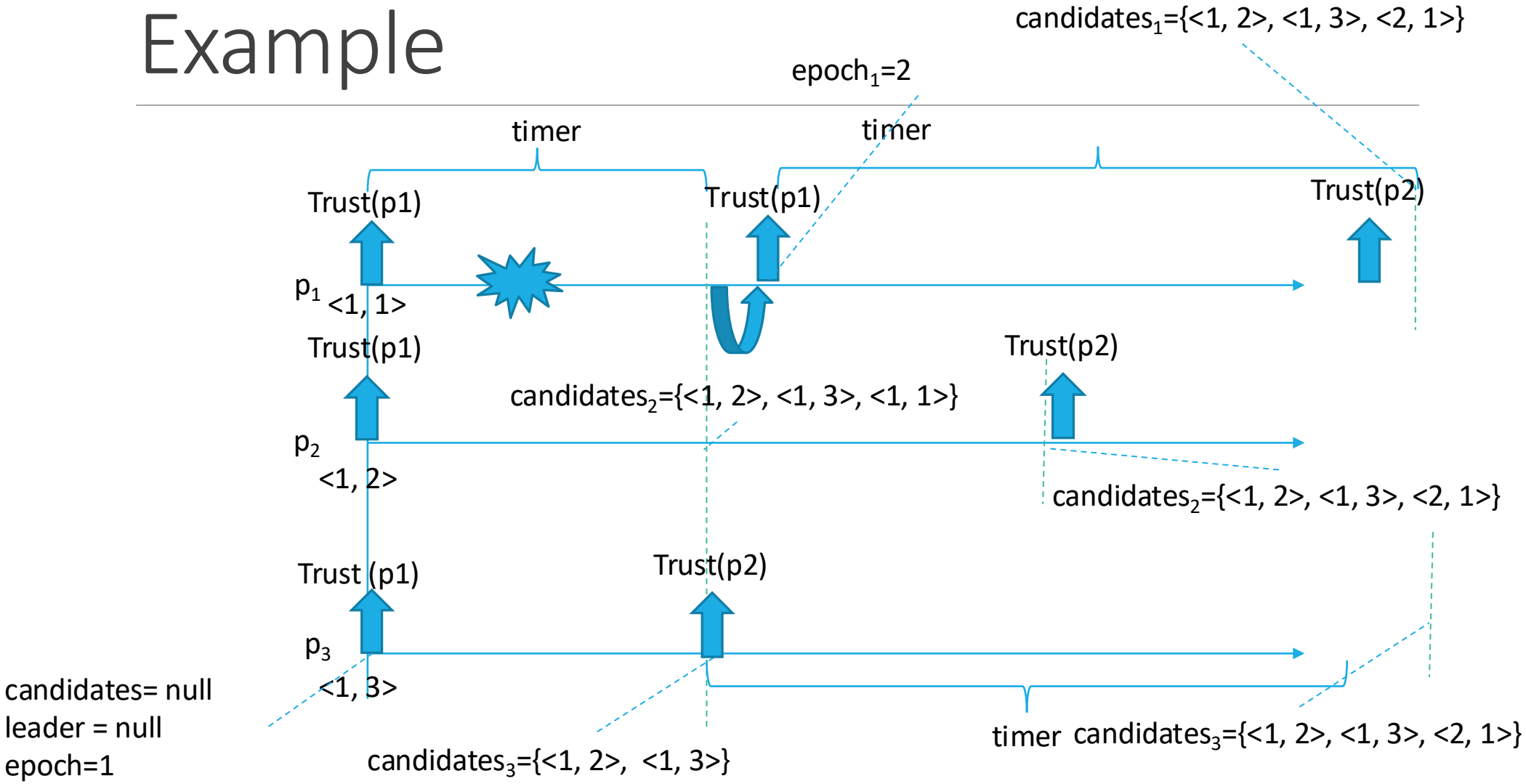
deterministic function returning one process among all candidates (i.e., process with the lowest epoch number and among the ones with the same epoch number the one with the lowest identifier)



Example


timer
timer
Trust(p1)
candidates$_1$={<1, 2>}
candidates$_1$={<1, 2>, <1, 3>, <1, 1>}
p_1
<1, 1>
<1, 1>
Trust(p1)
candidates$_2$={<1, 2>, <1, 3>, <1, 1>}
p_2
<1, 2>
Trust (p2)
Trust (p1)
Trust (p1)
timer
p_3
<1, 3>
candidates= null
leader = null
epoch=1
candidates$_3$={<1, 2>, <1, 3>}
Leader = p2
candidates$_3$={<1,1>, <1, 2>, <1, 3>}
Leader = p2

Example


candidates$_1$={<1, 2>, <1, 3>, <2, 1>}
epoch$_1$=2
timer
timer
Trust(p1)
Trust(p1)
Trust(p2)
p$_1$ <1, 1>
Trust(p1)
Trust(p2)
candidates$_2$={<1, 2>, <1, 3>, <1, 1>}
p$_2$
<1, 2>
candidates$_2$={<1, 2>, <1, 3>, <2, 1>}
Trust (p1)
Trust(p2)
p$_3$
<1, 3>
candidates= null
leader = null
epoch=1
candidates$_3$={<1, 2>, <1, 3>}
timer
candidates$_3$={<1, 2>, <1, 3>, <2, 1>}

References

C. Cachin, R. Guerraoui and L. Rodrigues. Introduction to Reliable and Secure Distributed Programming, Springer, 2011

- Chapter 2 – from Section 2.6.1 to Section 2.6.5